you use dependency injection.

7.1.4 Dependency injection

Dependency injection is a technique that helps us to achieve dependency inversion. As its name suggests, you inject the dependency into the consuming module, typically via its constructor as a parameter, and assign it to a private field for use within the consuming class. Injecting dependencies as arguments to a constructor method, as you will recall, helps us conform to the explicit dependencies principle.

The following code listing shows the `CreateModel`, altered to include an explicit constructor that takes an `ICityService` as a parameter. It assigns it to a private field, so it can be referenced within the class where needed.

Listing 7.4 Injecting `ICityService` dependency via `CreateModel`'s constructor

```
public class CreateModel : PageModel
{
    private readonly ICityService _cityService;     ❶
    public CreateModel(ICityService cityService)    ❷
    {
        _cityService = cityService;                 ❸
    }
    ...
}
```

❶ Add a private field to the class to store the dependency.

❷ Inject the dependency via the constructor.

❸ Assign the injected dependency to the private field.

Now your dependency is explicit. The collaborating object (any type that implements the `ICityService` interface) needed by your